Maths Library

This library allows for both ‘integer maths’ and ‘floating point maths’, with the routines named ‘imath_xxx’ and ‘fmath_xxx’ respectively.

The following routines are available in the library:

Name	Address	Description	Inputs	Outputs
imath_abs	0x2000	Absolute Value	R0: x	R0: abs(x)
imath_pow	0x2010	Exponentiation	R0: x R1: n	R0: x^n
imath_mod	0x2040	Modulus	R0: x R1: y	R0: x % y
imath_fact	0x2050	Factorial	R0: x	R0: x!
imath_sqrt	0x2070	Integer Square Root	R0: x	R0: isqrt(x)
imath_vec_add	0x21b0	Vector Addition (c = a + b)	R0: addr of a R1: addr of b R2: addr of c	-
imath_vec_sub	0x21f0	Vector Subtraction (c = a - b)	R0: addr of a R1: addr of b R2: addr of c	-
imath_vec_sc_mul	0x2230	Vector Scalar Multiplication (b = m * a)	R0: addr of a R1: m R2: addr of b	-
imath_vec_dot_prod	0x2260	Dot Product	R0: addr of a R1: addr of b	R0: a.b
imath_mat_add	0x22a0	Matrix Addition (C = A + B)	R0: addr of A R1: addr of B R2: addr of C	-
imath_mat_sub	0x22e0	Matrix Subtraction (C = A - B)	R0: addr of A R1: addr of B R2: addr of C	-
imath_mat_sc_mul	0x2320	Matrix Scalar Multiplication (B = m * A)	R0: addr of A R1: m R2: addr of B	-
imath_mat_mul	0x2350	Matrix Multiplication (C = AB)	R0: addr of A R1: addr of B R2: addr of C	-

imath_mat_trans	0x2510	Matrix Transposition ($B = A^T$)	R0: addr of A R1: addr of B	-
imath_mat_det_2	0x2550	Matrix Determinant (2x2)	R0: addr of A	R0: |A|
imath_mat_det_3	0x2590	Matrix Determinant (3x3)	R0: addr of A	R0: |A|
imath_cmplx_re	0x2620	Complex Number Real Part	R0: addr of x	R0: Re(x)
imath_cmplx_im	0x2630	Complex Number Imaginary Part	R0: addr of x	R0: Im(x)
imath_cmplx_conj	0x2640	Complex Number Conjugate ($y = x^*$)	R0: addr of x R1: addr of y	-
imath_cmplx_add	0x2660	Complex Number Addition (z = x + y)	R0: addr of x R1: addr of y R2: addr of z	-
imath_cmplx_sub	0x2680	Complex Number Subtraction (z = x - y)	R0: addr of x R1: addr of y R2: addr of z	-
imath_cmplx_sc_mul	0x26a0	Complex Number Scalar Multiplication (y = m * x)	R0: addr of x R1: m R2: addr of y	-
imath_cmplx_mul	0x26c0	Complex Number Multiplication (z = x * y)	R0: addr of x R1: addr of y R2: addr of z	-
fmath_abs	0x26f0	Absolute Value	R0: x	R0: abs(x)
fmath_pow	0x2700	Exponentiation	R0: x R1: n (int)	R0: x^n
fmath_sqrt	0x2740	Square Root	R0: x	R0: sqrt(x)
fmath_sin	0x2750	Sin : x in degs	R0: x	R0: sin(x)
fmath_cos	0x27a0	Cos : x in degs	R0: x	R0: cos(x)
fmath_tan	0x27f0	Tan : x in degs	R0: x	R0: tan(x)
fmath_vec_add	0x2840	Vector Addition (c = a + b)	R0: addr of a R1: addr of b R2: addr of c	-
fmath_vec_sub	0x2880	Vector Subtraction (c = a - b)	R0: addr of a R1: addr of b R2: addr of c	-
fmath_vec_sc_mul	0x28c0	Vector Scalar Multiplication (b = m * a)	R0: addr of a R1: m R2: addr of b	-

fmath_vec_dot_prod	0x28f0	Dot Product	R0: addr of a R1: addr of b	R0: a.b
fmath_mat_add	0x2930	Matrix Addition (C = A + B)	R0: addr of A R1: addr of B R2: addr of C	-
fmath_mat_sub	0x2970	Matrix Subtraction (C = A - B)	R0: addr of A R1: addr of B R2: addr of C	-
fmath_mat_sc_mul	0x29b0	Matrix Scalar Multiplication (B = m * A)	R0: addr of A R1: m R2: addr of B	-
fmath_mat_mul	0x29e0	Matrix Multiplication (C = AB)	R0: addr of A R1: addr of B R2: addr of C	-
fmath_mat_trans	0x2bf0	Matrix Transposition ($B = A^T$)	R0: addr of A R1: addr of B	-
fmath_mat_det_2	0x2c10	Matrix Determinant (2x2)	R0: addr of A	R0: |A|
fmath_mat_det_3	0x2c50	Matrix Determinant (3x3)	R0: addr of A	R0: |A|
fmath_cmplx_re	0x2ce0	Complex Number Real Part	R0: addr of x	R0: Re(x)
fmath_cmplx_im	0x2cf0	Complex Number Imaginary Part	R0: addr of x	R0: Im(x)
fmath_cmplx_conj	0x2d00	Complex Number Conjugate ($y = x^*$)	R0: addr of x R1: addr of y	-
fmath_cmplx_add	0x2d20	Complex Number Addition (z = x + y)	R0: addr of x R1: addr of y R2: addr of z	-
fmath_cmplx_sub	0x2d50	Complex Number Subtraction (z = x - y)	R0: addr of x R1: addr of y R2: addr of z	-
fmath_cmplx_sc_mul	0x2d80	Complex Number Scalar Multiplication (y = m * x)	R0: addr of x R1: m R2: addr of y	-
fmath_cmplx_mul	0x2da0	Complex Number Multiplication (z = x * y)	R0: addr of x R1: addr of y R2: addr of z	-
fmath_exp	0x2dd0	Exponentiation NB: Valid results for inputs in range (-2.5,2.5)	R0: x	R0: e^x

fmath_ln	0x2e10	Natural Log NB: Valid results for inputs in range (0.05,5.0)	R0: x	R0: ln(x)

Any programs using these routines should include the following definitions:

```
.equ imath_abs          0x2000
.equ imath_pow          0x2010
.equ imath_mod          0x2040
.equ imath_fact         0x2050
.equ imath_sqrt         0x2070

.equ imath_vec_add      0x21b0
.equ imath_vec_sub      0x21f0
.equ imath_vec_sc_mul   0x2230
.equ imath_vec_dot_prod 0x2260

.equ imath_mat_add      0x22a0
.equ imath_mat_sub      0x22e0
.equ imath_mat_sc_mul   0x2320
.equ imath_mat_mul      0x2350
.equ imath_mat_trans    0x2510
.equ imath_mat_det_2    0x2550
.equ imath_mat_det_3    0x2590

.equ imath_cmplx_re     0x2620
.equ imath_cmplx_im     0x2630
.equ imath_cmplx_conj   0x2640
.equ imath_cmplx_add    0x2660
.equ imath_cmplx_sub    0x2680
.equ imath_cmplx_sc_mul 0x26a0
.equ imath_cmplx_mul    0x26c0
```

```
.equ MATH_PI      3.141593
.equ MATH_E       2.718282
.equ MATH_DEG_RAD 0.017453

.equ MATH_NAN     0x7e00
.equ MATH_INF_POS 0x7c00
.equ MATH_INF_NEG 0xfc00
```

```
.equ fmath_abs          0x26f0
.equ fmath_pow          0x2700
.equ fmath_sqrt         0x2740
.equ fmath_sin          0x2750
.equ fmath_cos          0x27a0
.equ fmath_tan          0x27f0

.equ fmath_vec_add      0x2840
.equ fmath_vec_sub      0x2880
.equ fmath_vec_sc_mul   0x28c0
.equ fmath_vec_dot_prod 0x28f0

.equ fmath_mat_add      0x2930
.equ fmath_mat_sub      0x2970
.equ fmath_mat_sc_mul   0x29b0
.equ fmath_mat_mul      0x29e0
.equ fmath_mat_trans    0x2bf0
.equ fmath_mat_det_2    0x2c10
.equ fmath_mat_det_3    0x2c50

.equ fmath_cmplx_re     0x2ce0
.equ fmath_cmplx_im     0x2cf0
.equ fmath_cmplx_conj   0x2d00
.equ fmath_cmplx_add    0x2d20
.equ fmath_cmplx_sub    0x2d50
.equ fmath_cmplx_sc_mul 0x2d80
.equ fmath_cmplx_mul    0x2da0

.equ fmath_exp          0x2dd0
.equ fmath_ln           0x2e10
```

When using these routines in ‘ASM’ code the ‘Input Registers’ should be set up, then a ‘call func’ instruction used, with any result then being available in R0 following the ‘call’.

And when using ‘RMG’ code the ‘Input Registers’ are used as arguments to the function call, with any result being assigned as required.

Here are ‘ASM’ and ‘RMG’ examples of ‘imath_pow’ being used:

```
ldi r0,4            ; r0 -> 4^3 = 64
ldi r1,3
call imath_pow
```

```
res = imath_pow(4, 3);  // res = 4^3 = 64
```